



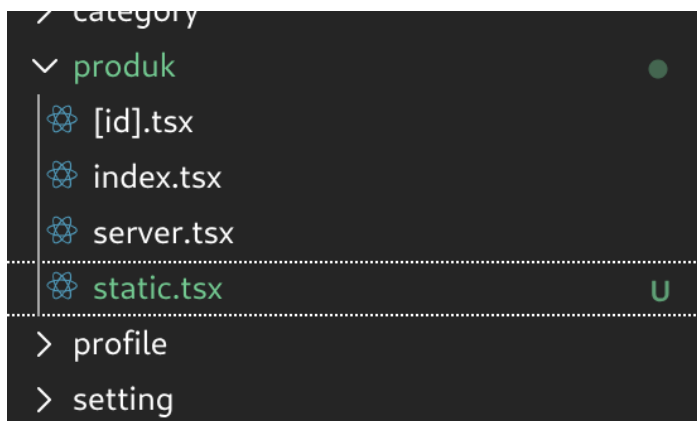
Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 9

Laporan Jobsheet

Praktikum 1

1.



```
1 import TampilanProduk from "../views/produk";
2 import { ProductType } from "../types/Product.types";
3
4 export default function HalamanprodukStatic (props:{products:ProductType[]}){
5   const { products } = props;
6
7   return(
8     <div>
9       <h1>Halaman Produk Statik</h1>
10      <TampilanProduk products={products}/>
11    </div>
12  )
13 }
14
15 export async function getStaticProps() {
16   const res = await fetch('http://localhost:3000/api/produk');
17   // const products: ProductType[] = await res.json();
18   const response : {data: ProductType[]} = await res.json();
19   return {
20     props: {
21       products: response.data
```



```
object {3}
  status : true
  status_code : 200
  data [2]
    0 {6}
      id : "9mIrRd07IIEUD1r0gsBX"
      size : "XL"
      category : "Baju Pria"
      price : 150000
      images : "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRs_dA4xde-4ioPFv6CmiVBWkoRXhIvC_iFJg&s"
      name : "Baju Model"
    1 {6}
      id : "DUgF726E1JRMG7ZNM8sQ"
      price : 10000
      category : "Baju Anak"
      size : "XL"
      name : "Baju polo"
      images : "https://www.johnbullclothing.com/cdn/shop/products/kombat-kids-army-tank-suit-btp-john-bull-clothing-243499.jpg?v=1718720821&width=1200"
```

- Menambahkan elemen statik pada halaman produk mirip seperti ssr namun bersifat statik

```
✓ Generating static pages using 31 workers (17/17) in 357.1ms
✓ Finalizing page optimization in 7.4ms

Route (pages)
├─ ○ /
├─ /_app
├─ ○ /404
├─ ○ /about
├─ f /api/hello
├─ f /api/produk
├─ ○ /auth/login
├─ ○ /auth/register
├─ ○ /blog/[slug]
├─ ○ /blog/blog
├─ ○ /category/[[...slug]]
├─ ○ /produk
├─ ○ /produk/[id]
├─ f /produk/server
├─ ● /produk/static
├─ ○ /profile/edit
├─ ○ /setting/app
├─ ○ /setting/user/password
├─ ○ /shop/[[...slug]]
├─ ○ /user
├─ ○ (Static) prerendered as static content
├─ ● (SSG) prerendered as static HTML (uses getStaticProps)
├─ f (Dynamic) server-rendered on demand

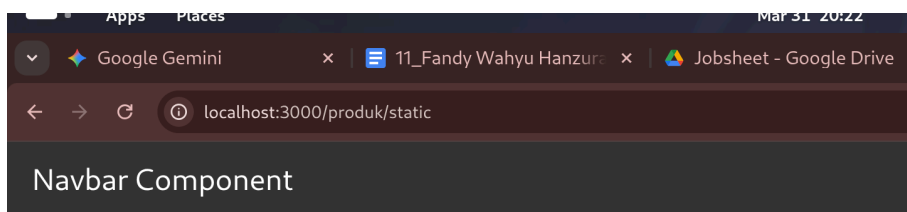
2. ○ (fandy🐙 dock) - [~/Downloads/git/PemrogramanFramework/pertemuan02/my-app]
```



```
(fandy@dock) - [~/Downloads/git/PemrogramanFramework/pertemuan02/my-app]
$ npm run start

> my-app@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://192.168.110.182:3000
```

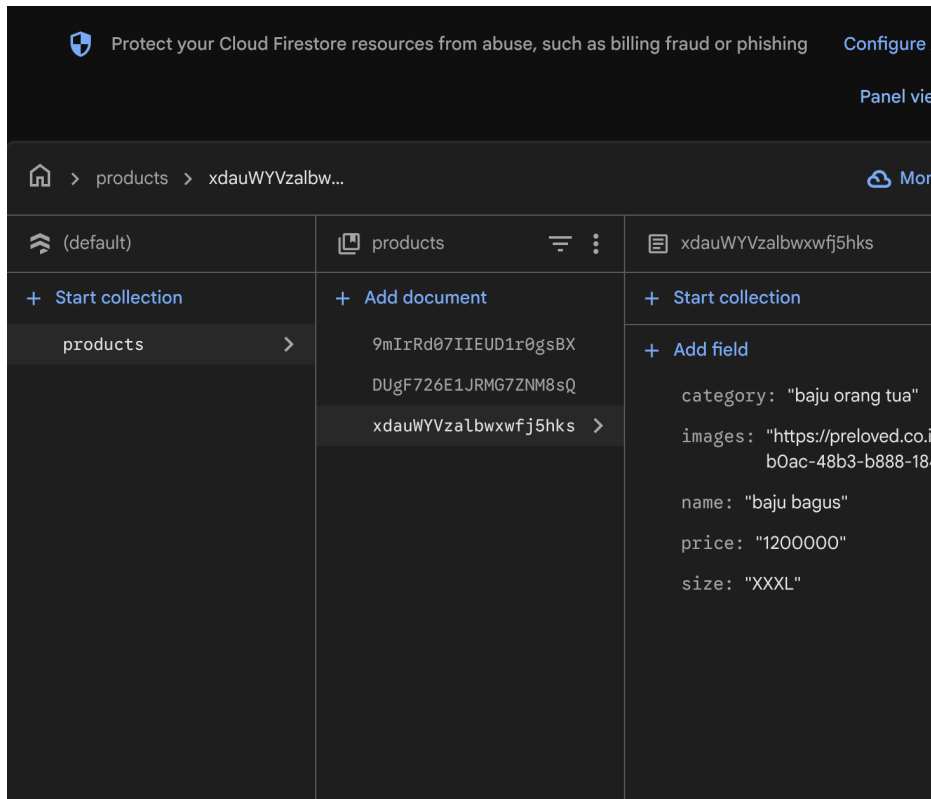


Halaman Produk Statik

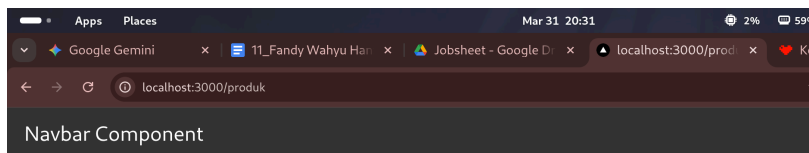
Daftar Produ

 Baju Model Baju Pria Rp 150.000	 Baju polo Baju Anak Rp 10.000
--	--

- Menerapkan build mode dengan menjalankan npm run build setelah berhasil ter eksekusi maka akan dilakukan npm run start untuk menjalankan server

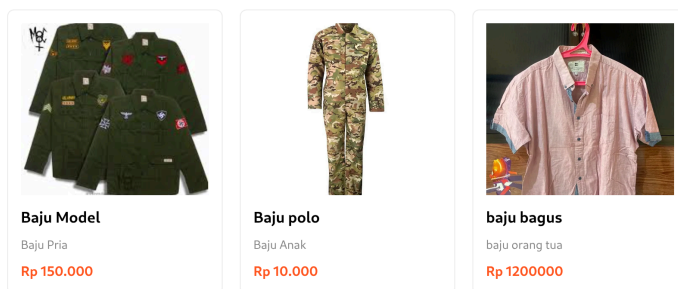


3.



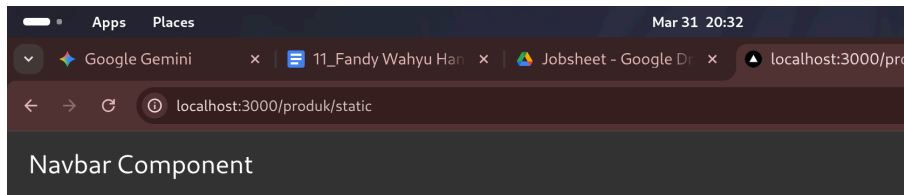
Refresh Data Barang

Daftar Produk



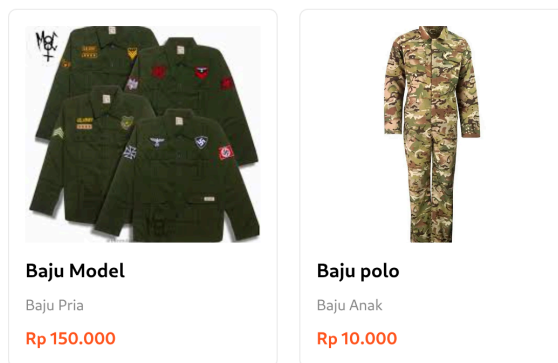
Yo its me Fandy Wahyu Hanzura

Yo its me Fandy Wahyu Hanzura



Halaman Produk Statik

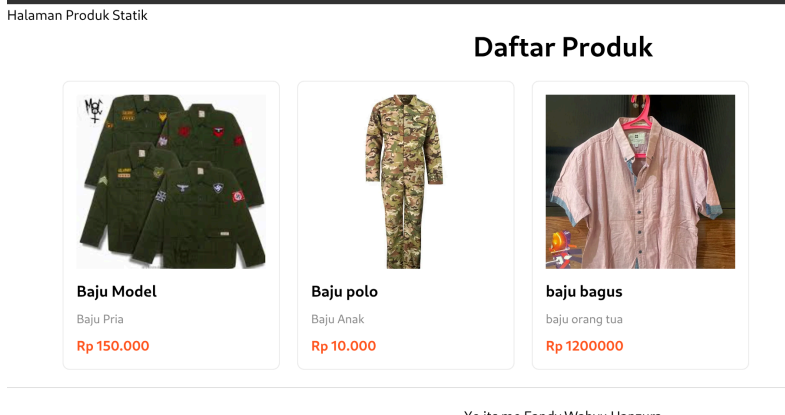
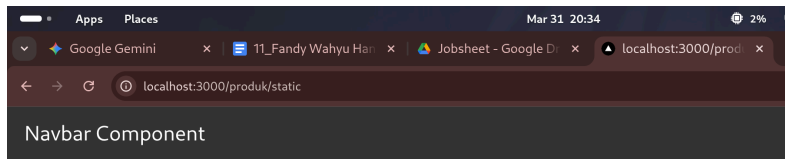
Daftar Produk



```
Route (pages)
  ○ /
  ○ /_app
  ○ /404
  ○ /about
  f /api/hello
  f /api/produk
  ○ /auth/login
  ○ /auth/register
  ○ /blog/[slug]
  ○ /blog/blog
  ○ /category/[...slug]
  ○ /produk
  ○ /produk/[id]
  f /produk/server
  ● /produk/static (729 ms)
  ○ /profile/edit
  ○ /setting/app
  ○ /setting/user/password
  ○ /shop/[...slug]
  ○ /user

  ○ (Static) prerendered as static content
  ● (SSG) prerendered as static HTML (uses getStaticProps)
  f (Dynamic) server-rendered on demand

(fandy@dock) - [~/Downloads/git/PemrogramanFramework/pertemuan02/my-app]
```



- melakukan penambahan data pada firebase ketika di refresh pada halaman npm run start maka akan menunjukkan bahwa halaman utama scr akan bertambah barang nya, namun pada halaman statik scr tidak menunjukkan penambahan sehingga perlu dilakukan build ulang dan restart ulang agar data pada halaman statik bertambah

Tugas Praktikum

Pemilihan strategi rendering dalam pengembangan aplikasi web modern bukan sekadar masalah teknis, melainkan sebuah keputusan arsitektural yang berdampak langsung pada pengalaman pengguna dan efisiensi sistem. Melalui rangkaian praktikum pada Jobsheet 10, terdapat tiga pilar utama yang dieksplorasi, yaitu Client-Side Rendering (CSR), Server-Side Rendering (SSR), dan Static Site Generation (SSG). Ketiganya memiliki karakteristik yang sangat berbeda, terutama dalam hal kapan dan di mana proses pengambilan data dilakukan.

Dalam implementasi Client-Side Rendering (CSR) yang dipadukan dengan library SWR, proses rendering sepenuhnya dilakukan di sisi browser setelah file HTML awal yang kosong diterima dari server. Fenomena ini memberikan fleksibilitas tinggi bagi aplikasi untuk selalu menampilkan data paling baru. Bukti nyata dari mekanisme ini terlihat pada tab Network, di mana muncul sebuah request fetch yang diinisiasi oleh browser untuk mengambil data dari endpoint API setelah halaman dimuat sepenuhnya. Selain itu, pengamatan pada tab Console menunjukkan adanya transisi data dari array kosong [] menjadi berisi objek produk. Karena adanya jeda waktu pengambilan data tersebut,



penggunaan Skeleton Loading dengan animasi berkedip menjadi sangat krusial sebagai feedback visual agar pengguna tidak merasa aplikasi sedang error saat layar terlihat kosong .

Perbedaan yang sangat kontras ditemukan saat menganalisis Server-Side Rendering (SSR). Pada strategi ini, proses pengambilan data dilakukan di sisi server sebelum HTML dikirimkan ke browser pengguna. Dampak teknisnya adalah data produk langsung tersedia secara utuh saat halaman pertama kali dibuka, tanpa melalui fase array kosong pada konsol browser. Analisis pada tab Network juga menunjukkan ketiadaan request fetch tambahan untuk konten utama karena datanya sudah "dijahit" langsung ke dalam dokumen HTML awal. SSR memberikan keuntungan besar pada aspek SEO dan kecepatan akses awal, karena konten sudah matang sejak detik pertama.

Selanjutnya, aspek Static Site Generation (SSG) memberikan dimensi analisis yang berbeda mengenai siklus hidup sebuah halaman web. SSG bekerja dengan cara meng-generate halaman menjadi file statis pada saat proses pembangunan atau build aplikasi dilakukan. Berdasarkan hasil pengamatan praktikum, terdapat perilaku unik pada sistem SSG murni. Ketika data baru ditambahkan ke dalam database (seperti Firebase), halaman web yang menggunakan SSG tidak akan menunjukkan perubahan data tersebut meskipun pengguna melakukan penyegaran halaman berkali-kali. Hal ini dikarenakan halaman tersebut sudah "terkunci" sebagai file statis pada saat proses build awal dilakukan.

Untuk menampilkan data terbaru pada halaman SSG, pengembang harus menjalankan ulang perintah build untuk menarik data terbaru dan membuat ulang file HTML statis yang baru. Karakteristik ini membuat SSG sangat efisien dalam hal kecepatan akses karena server hanya perlu mengirimkan file yang sudah jadi, namun sangat tidak efisien untuk aplikasi yang datanya berubah setiap saat seperti stok barang pada e-commerce atau dashboard monitoring real-time. Sebaliknya, CSR yang didukung oleh SWR menjadi solusi yang lebih dinamis karena fitur caching dan revalidation-nya memungkinkan data tetap diperbarui tanpa perlu melakukan proses pembangunan ulang aplikasi secara keseluruhan .

Sebagai kesimpulan dari analisis praktikum ini, setiap metode rendering memiliki peranan yang spesifik tergantung pada kebutuhan fungsional aplikasi. CSR sangat unggul dalam interaktivitas dan pembaruan data real-time, SSR sangat kuat dalam hal SEO dan kecepatan muat awal konten matang, sedangkan SSG menawarkan performa maksimal untuk konten yang jarang berubah. Pemahaman mendalam mengenai transisi status data pada konsol dan aktivitas jaringan pada tab network menjadi kompetensi utama dalam mendiagnosis perilaku aplikasi dan menentukan strategi rendering yang paling tepat guna mencapai performa sistem yang optimal serta pengalaman pengguna yang memuaskan

Pertanyaan refleksi

1. Mengapa SSG tidak menampilkan data terbaru?



- **Efisiensi Kode:** `useEffect` memerlukan pengelolaan *state* manual untuk loading, error, dan data, sedangkan `SWR` menyediakan status `isLoading`, error, dan data secara otomatis dalam satu baris kode.
- **Caching:** `useEffect` akan melakukan pengambilan data ulang dari nol setiap kali halaman di-render (menyebabkan kedipan/skeleton), sementara `SWR` memiliki fitur *caching* yang menampilkan data lama terlebih dahulu agar transisi terasa instan (*stale-while-revalidate*).
- **Fitur Otomatis:** `SWR` memiliki fitur pembaruan data otomatis saat jendela browser kembali difokuskan (*revalidate on focus*), yang tidak dimiliki oleh `useEffect` standar kecuali diprogram secara manual.

2. Mengapa SSG lebih cepat?

- **Menghindari Tampilan Kosong:** Karena pada CSR terdapat jeda (*delay*) antara pemuatan halaman dan selesainya proses *fetching* data, *skeleton* digunakan agar pengguna tidak melihat layar putih atau kosong.
- **Feedback Visual:** *Skeleton* memberikan indikasi visual kepada pengguna bahwa aplikasi sedang bekerja dan data sedang dalam proses pemuatan.
- **Meningkatkan UX:** Dengan *skeleton*, struktur tata letak (*layout*) sudah terbentuk lebih awal, sehingga tidak terjadi perubahan posisi elemen yang mendadak (*layout shift*) saat data muncul.

3. Kapan SSG tidak cocok digunakan?

- **Optimalisasi SEO:** SSR sebaiknya dipilih untuk halaman yang membutuhkan indeksasi mesin pencari yang kuat, karena server mengirimkan HTML dengan konten yang sudah lengkap.
- **Kecepatan Akses Awal:** SSR lebih baik digunakan jika ingin pengguna langsung menerima konten matang tanpa harus menunggu proses *loading* tambahan di sisi *client*.
- **Konten Statis atau Jarang Berubah:** Jika data tidak memerlukan interaktivitas tinggi di sisi klien, SSR memberikan performa yang lebih stabil karena beban pemrosesan ada di server.

4. Mengapa e-commerce tidak cocok menggunakan SSG murni?

- E-commerce memiliki data yang sangat dinamis, seperti **stok barang** dan **harga diskon** yang bisa berubah setiap detik.
- Pada SSG murni, halaman bersifat statis dan hanya diperbarui saat proses *build* ulang aplikasi dilakukan.
- Jika stok habis, halaman SSG akan tetap menampilkan "Tersedia" sampai server melakukan *re-build* seluruh aplikasi, yang tentu saja tidak efisien dan berisiko merugikan transaksi bisnis.



5. Apa perbedaan build mode dan development mode?

- **Request Fetch/XHR:** Pada CSR, akan terlihat adanya *request* tambahan (tipe **fetch**) ke endpoint API setelah halaman dimuat. Pada SSR, *request* data utama ini tidak terlihat karena sudah selesai di sisi server.
- **Isi Response HTML:** Jika melakukan "View Page Source", HTML pada CSR cenderung kosong di bagian bodi (*empty shell*), sedangkan pada SSR bodi HTML sudah berisi teks dan data produk secara lengkap.
- **Urutan Log Console:** Pada CSR, *log console* akan menunjukkan perubahan dari array kosong [] menjadi berisi data. Pada SSR, data langsung tersedia dalam bentuk objek utuh saat pertama kali halaman terbuka.